



10. CNN

≡ Topic

Mục lục

1. [Tại sao cần Convolution?](#)
2. [Định nghĩa Toán học](#)
3. [Thuật toán Tính Convolution](#)
4. [Multi-channel Input](#)
5. [Multi-filter Layer](#)
6. [Padding](#)
7. [Strides](#)
8. [Công thức Kích thước Tổng quát](#)
9. [Pooling Layer](#)
10. [Tổng kết](#)

1. Tại sao cần Convolution?

1.1 Vấn đề của Fully Connected trên ảnh

Hãy tưởng tượng ta áp MLP thuần túy lên một ảnh $224 \times 224 \times 3$ (khoảng 150k pixel). Lớp FC đầu tiên với 1024 neuron sẽ cần $150,528 \times 1,024 \approx 154$ triệu tham số — chỉ cho một lớp! Điều này gây ra hai vấn đề nghiêm trọng:

1. **Quá nhiều tham số** → overfit nặng, đòi hỏi dữ liệu khổng lồ
2. **Phá vỡ cấu trúc không gian**: FC xử lý tất cả pixel như nhau, không biết pixel (1,1) và pixel (1,2) ở cạnh nhau. Mèo vẫn là mèo dù nằm bên trái hay bên phải ảnh.

1.2 Convolution giải quyết bằng hai nguyên lý

- **Local connectivity (Kết nối cục bộ)**: Mỗi neuron chỉ nhìn vào một vùng nhỏ (receptive field) của input, không phải toàn bộ ảnh.
- **Weight sharing (Chia sẻ trọng số)**: Cùng một bộ filter được áp lên **tất cả** vị trí trên ảnh — nếu filter phát hiện cạnh ngang ở góc trên trái, nó cũng phát hiện cạnh ngang ở góc dưới phải với cùng trọng số.

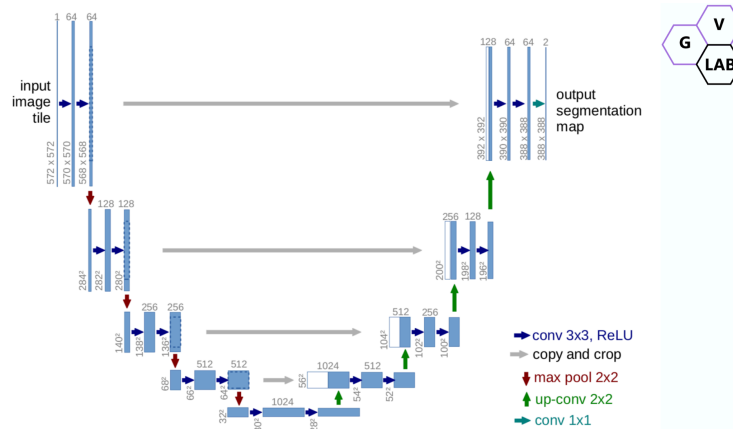
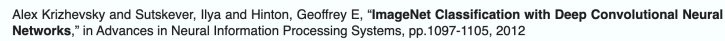
Phép ẩn dụ: Convolution giống như dùng một cái kính lúp (filter/kernel) để quét khắp bức ảnh. Cái kính lúp có cùng hình dạng và trọng số tại mọi vị trí — nó tìm kiếm cùng một "đặc trưng cục bộ" ở khắp nơi.

1.3 Convolution trong thực tế Deep Learning

Các kiến trúc CNN kinh điển đã cách mạng hóa Computer Vision:

Kiến trúc	Năm	Đặc điểm
AlexNet	2012	Conv-Norm-Pool lặp lại → FC → Softmax; đánh bại các phương pháp truyền thống tại ImageNet
VGG-16	2014	Chống nhiễu lớp 3×3 Conv nhỏ thay vì Conv lớn → sâu hơn, tốt hơn
UNet	2015	Encoder-Decoder với skip connections; dùng cho phân đoạn ảnh y tế

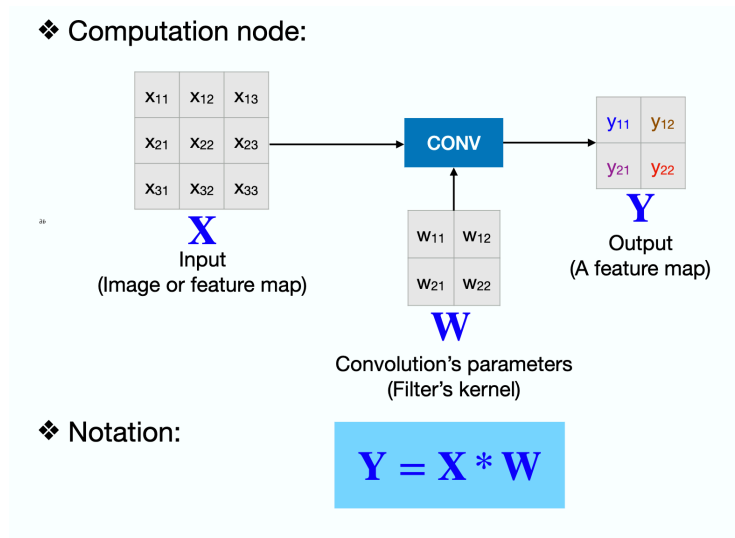
Điểm chung: **Convolution biến đổi pixel thô thành biểu diễn có ý nghĩa** — từng lớp học đặc trưng ngày càng trừu tượng hơn.



Olaf_Ronneberger, Philipp_Fischer, Thomas_Brox, "U-Net: Convolutional Networks for Biomedical Image Segmentation," arXiv:1505.04597v1 [cs.CV]

2. Định nghĩa Toán học

2.1 Ký hiệu



$X \rightarrow W \rightarrow Y = X * W$
(Input) (Filter/Kernel) (Output/Feature map)

- X : ảnh đầu vào hoặc feature map (kích thước $i_1 \times i_2$)
- W : kernel/filter của convolution (kích thước $k \times k$, với bán kính $r = \lfloor k/2 \rfloor$)
- Y : feature map đầu ra
- : **KHÔNG** phải nhân ma trận — đây là phép tích chập

2.2 Convolution vs Cross-Correlation

Đây là hai phép toán khác nhau nhưng thường bị nhầm lẫn:

Convolution (Tích chập thực sự):

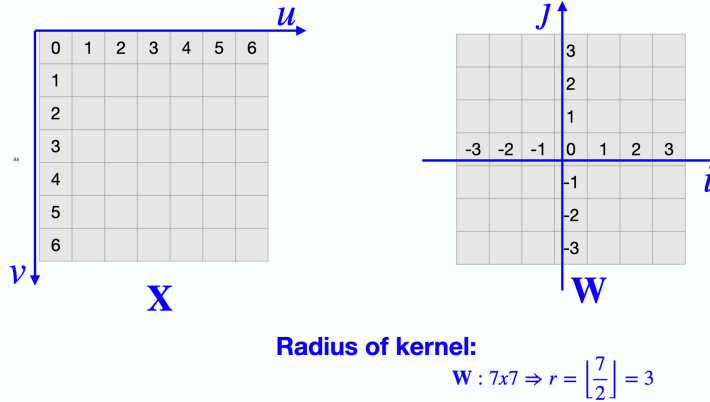
$$Y(u, v) = X * W = \sum_{i=-r}^r \sum_{j=-r}^r X(u-i, v-j) \cdot W(i, j)$$

Cross-Correlation (Tương quan chéo):

$$Y(u, v) = X \star W = \sum_{i=-r}^r \sum_{j=-r}^r X(u+i, v+j) \cdot W(i, j)$$

Sự khác biệt: Convolution dùng $(u-i, v-j)$ — tức là **flip kernel 180°** trước khi tính dot product. Cross-correlation không flip.

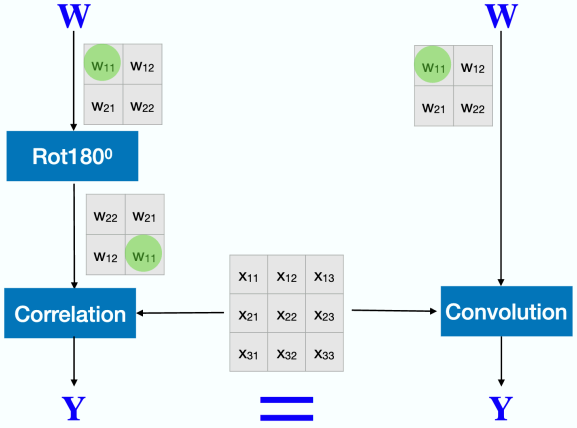
❖ Definition:



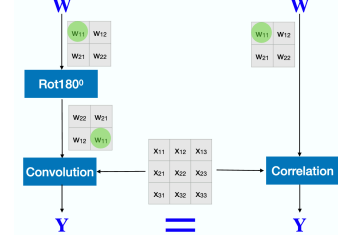
Quan hệ: Convolution với kernel W = Cross-correlation với kernel $\text{Rot}_{180^\circ}(W)$.

Trong Deep Learning thực tế: Hầu hết framework (PyTorch, TensorFlow) thực ra triển khai **cross-correlation** và gọi nó là "convolution". Điều này không ảnh hưởng vì kernel là tham số học được — nếu cần flip, mạng sẽ tự học kernel đã flipped.

Convolution vs cross-correlation:



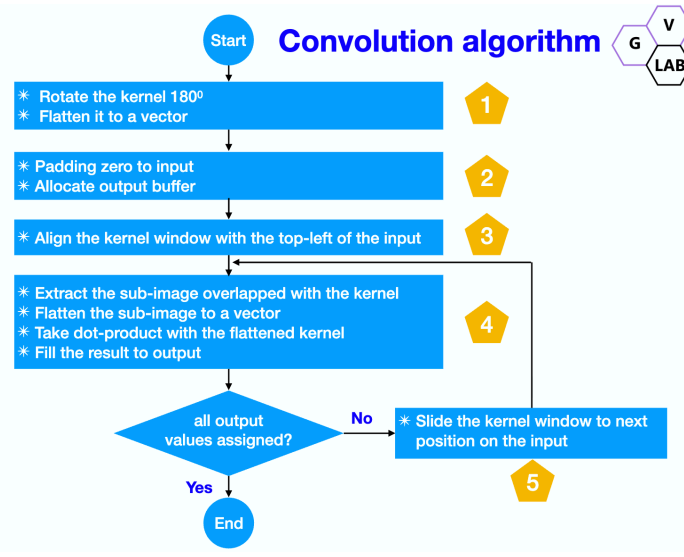
Convolution vs cross-correlation:



[Cần ành]: Sơ đồ so sánh Convolution vs Cross-Correlation: kernel W , kernel $\text{Rot}_{180^\circ}(W)$, kết quả Y tương ứng.

3. Thuật toán Tính Convolution

3.1 Quy trình step-by-step



Bước 1: Rotate kernel 180° (flip ngang + flip dọc)
 Bước 2: Flatten kernel thành vector
 Bước 3: Padding zero vào input (nếu cần)
 Bước 4: Allocate output buffer
 Bước 5: Align kernel với góc trên-trái của input

```

graph TD
    B5 --> L1[Lấy sub-image (vùng input mà kernel che phủ)]
    L1 --> L2[Flatten sub-image thành vector]
    L2 --> L3[Dot product với kernel đã flatten]
    L3 --> L4[Ghi kết quả vào output]
    L4 --> L5[Slide kernel sang vị trí tiếp theo]
    L5 --> L6[Lặp cho đến khi đã gán hết output]
  
```

3.2 Ví dụ số cụ thể (từ slide)

Input 4×4 , Kernel 3×3 , No padding, Stride = 1:

$$X = \begin{bmatrix} 3 & 1 & 0 & 1 \\ 1 & 1 & 2 & 0 \\ 1 & 2 & 2 & 1 \\ 0 & 1 & 0 & 2 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 & 2 \\ 1 & 2 & 0 \\ 0 & 1 & 1 \end{bmatrix}$$

Bước 1: Rotate W 180°:

$$\text{Rot}_{180^\circ}(W) = \begin{bmatrix} 1 & 1 & 0 \\ 0 & 2 & 1 \\ 2 & 0 & 1 \end{bmatrix}$$

Bước 2: Flatten: $[1, 1, 0, 0, 2, 1, 2, 0, 1]$

Bước 3–5: Tính dot product tại từng vị trí (slide từng bước):

Vị trí	Sub-image (flatten)	Dot product	Kết quả
(1,1) — top-left	$[3, 1, 0, 1, 1, 2, 1, 2, 2]$	$3 \times 1 + 1 \times 1 + 0 \times 0 + 1 \times 0 + 1 \times 2 + 2 \times 1 + 1 \times 2 + 2 \times 0 + 2 \times 1$	12
(1,2)	$[1, 0, 1, 1, 2, 0, 2, 2, 1]$	$1 \times 1 + 0 \times 1 + 1 \times 0 + 1 \times 0 + 2 \times 2 + 0 \times 1 + 2 \times 2 + 2 \times 0 + 1 \times 1$	10
(2,1)	$[1, 1, 2, 2, 2, 1, 0, 1, 0]$	...	8
(2,2)	$[1, 2, 0, 2, 2, 1, 1, 0, 2]$	...	12

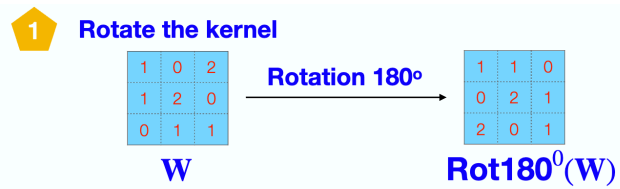
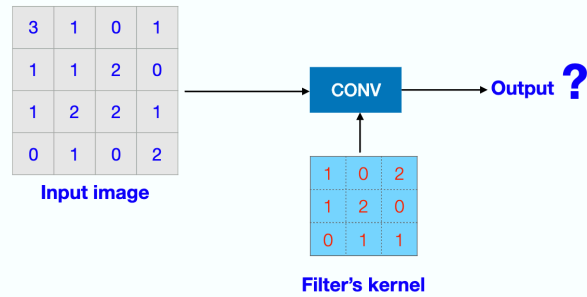
Kết quả:

$$Y = \begin{bmatrix} 12 & 10 \\ 8 & 12 \end{bmatrix}$$

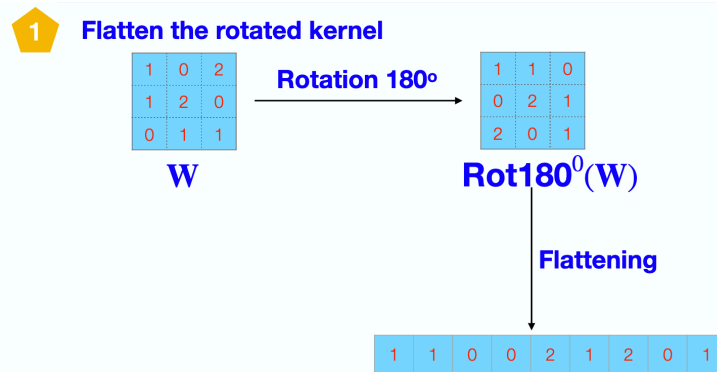
Công thức kích thước output (no padding, stride=1):

$$o_1 \times o_2 = (i_1 - k_1 + 1) \times (i_2 - k_2 + 1)$$

Ví dụ: 4×4 input, 3×3 kernel $\rightarrow (4 - 3 + 1) \times (4 - 3 + 1) = 2 \times 2$ output.



$\text{Rot180}^0(W) = \text{Flip on horizontal direction} + \text{Flip on vertical direction}$



2 Padding the input

3	1	0	1
1	1	2	0
1	2	2	1
0	1	0	2

X

This section illustrates convolution with no padding, stride = 1



Y
Output

Output size: 2x2 will be clear shortly

3 Align the rotated kernel with the input image

3	1	0	1
1	1	2	0
1	2	2	1
0	1	0	2

Input image



1x3	1x1	0x0	1
0x1	2x1	1x2	0
2x1	0x2	1x2	1
0	1	0	2

1	1	0
0	2	1
2	0	1

Rot180⁰(W)

4 Compute dot-product

X and W: aligned at left-top

Get sub-image

1x3	1x1	0x0	1
0x1	2x1	1x2	0
2x1	0x2	1x2	1
0	1	0	2

Input image

1	1	0
0	2	1
2	0	1

Rot180⁰(W)

3	1	0
1	1	2
1	2	2

flattening

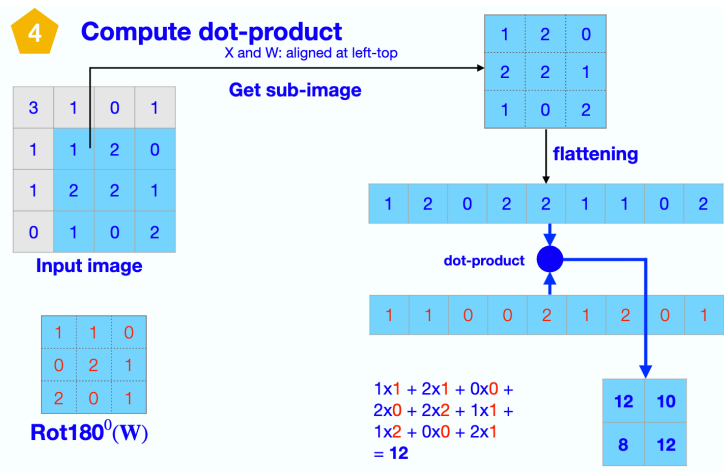
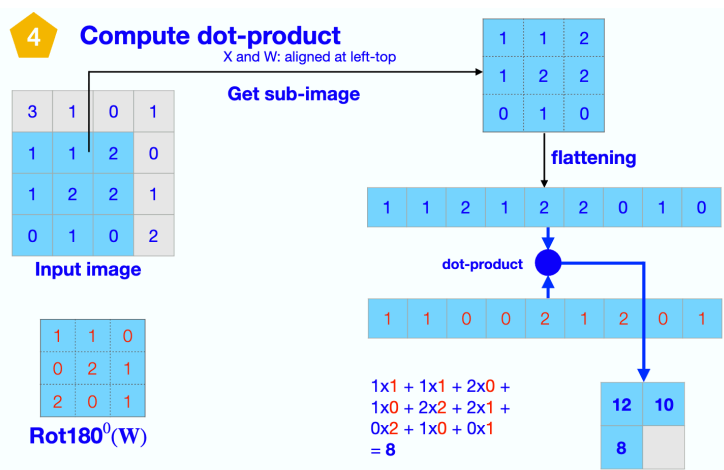
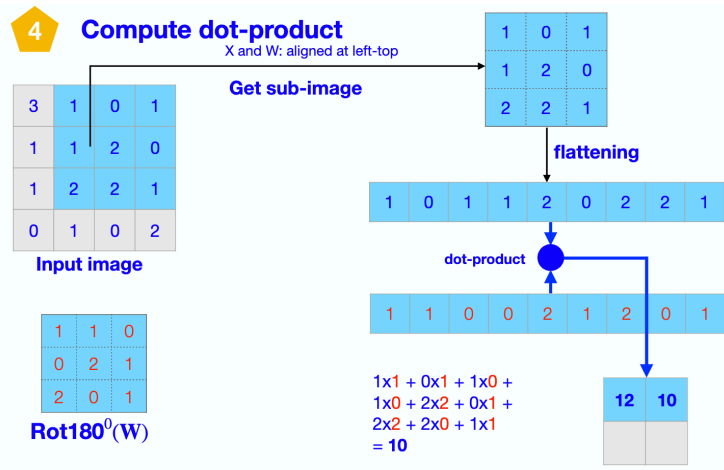
3	1	0	1	1	2	1	2	2
---	---	---	---	---	---	---	---	---

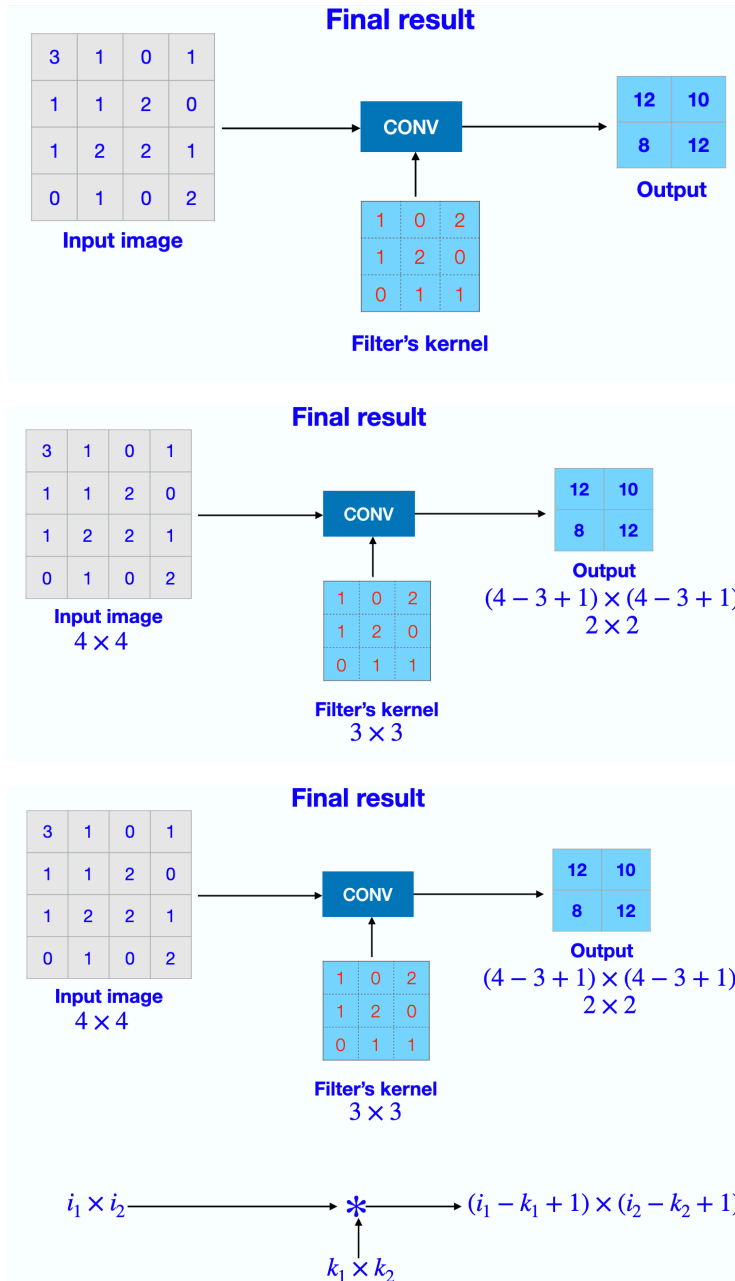
dot-product

1	1	0	0	2	1	2	0	1
---	---	---	---	---	---	---	---	---

$$\begin{aligned}
 &3 \times 1 + 1 \times 1 + 0 \times 0 + \\
 &1 \times 0 + 1 \times 2 + 2 \times 1 + \\
 &1 \times 2 + 2 \times 0 + 2 \times 1 \\
 &= 12
 \end{aligned}$$

12	





[Cảnh ảnh]: Animation hoặc sơ đồ step-by-step: kernel trượt qua input, highlight sub-image và kết quả tại mỗi vị trí. (Trang 21–30 slide Conv2d-intro)

4. Multi-channel Input

4.1 Ảnh thực tế có nhiều channels

Ảnh màu RGB có **3 channels**: Red, Green, Blue. Feature map từ lớp trước cũng có nhiều channels. Vậy convolution hoạt động như thế nào với multi-channel input?

Quy tắc: Kernel cũng phải có **cùng số channels** với input.

- Input: $i_1 \times i_2 \times D$ (D channels)
- Kernel: $k_1 \times k_2 \times D$ (cùng D channels)

4.2 Cách tính

1. Rotate từng channel của kernel 180° (mỗi channel riêng biệt)

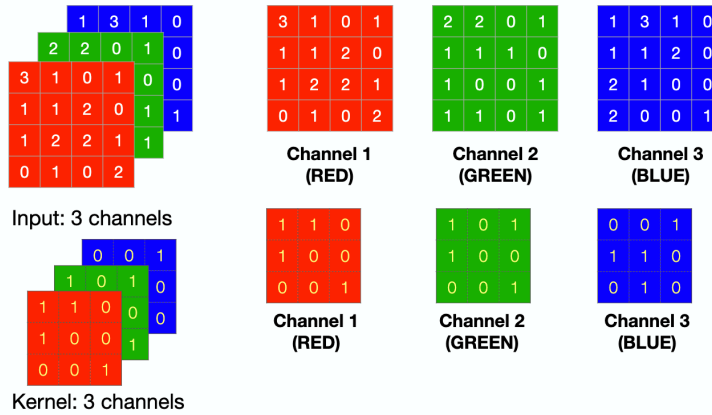
2. Flatten **toàn bộ** kernel (tất cả D channels) thành một vector dài
3. Flatten sub-image tương ứng (tất cả D channels) thành một vector dài
4. **Dot product** giữa hai vector dài → một giá trị scalar

$$Y(u, v) = \sum_{c=1}^D \sum_{i=-r}^r \sum_{j=-r}^r X_c(u+i, v+j) \cdot W_c(i, j)$$

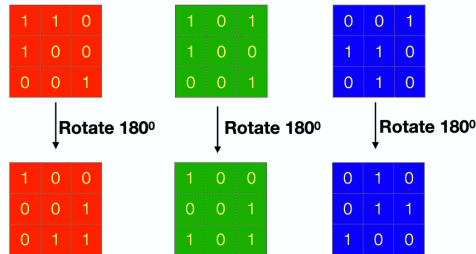
Điểm mấu chốt: Dù input có D channels, **output vẫn là 1 channel** khi dùng một filter. Một filter "tổng hợp" thông tin từ tất cả channels thành một feature map duy nhất.

Phép ẩn dụ: Tưởng tượng bạn là thám tử nhìn ảnh qua ba kính lọc màu (R, G, B) đồng thời. Mỗi kính cho thấy một khía cạnh khác nhau. Cuối cùng, bạn kết hợp thông tin từ cả ba và đưa ra một kết luận ("có cảnh ở vị trí này" — đó là feature map).

Input image or feature map: multiple channels



1 Flatten the rotated kernel to a vector



After flattening:



2 Padding the input



(No padding)

2 Allocate the output buffer

3	1	0	1
1	1	2	0
1	2	2	1
0	1	0	2

Channel 1
(RED)

2	2	0	1
1	1	1	0
1	0	0	1
1	1	0	1

Channel 2
(GREEN)

1	3	1	0
1	1	2	0
2	1	0	0
2	0	0	1

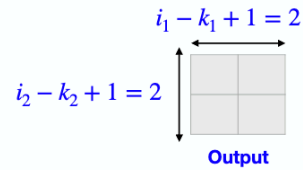
Channel 3
(BLUE)

input: $i_1 = i_2 = 4$

kernel: $k_1 = k_2 = 3$

padding: $p_1 = p_2 = 0$

strides: $s_1 = s_2 = 1$



3	1	0	1
1	1	2	0
1	2	2	1
0	1	0	2

2	2	0	1
1	1	1	0
1	0	0	1
1	1	0	1

1	3	1	0
1	1	2	0
2	1	0	0
2	0	0	1

3	1	0	1
1	1	2	0
1	2	2	1
0	1	0	2

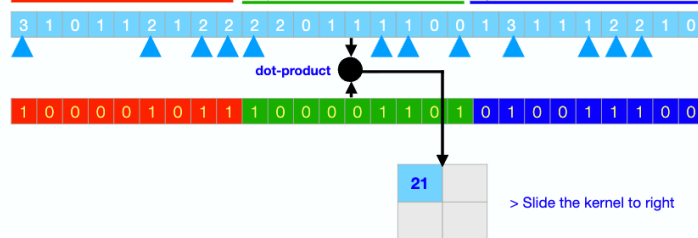
3	1	0
1	1	2
1	2	2
1	2	2

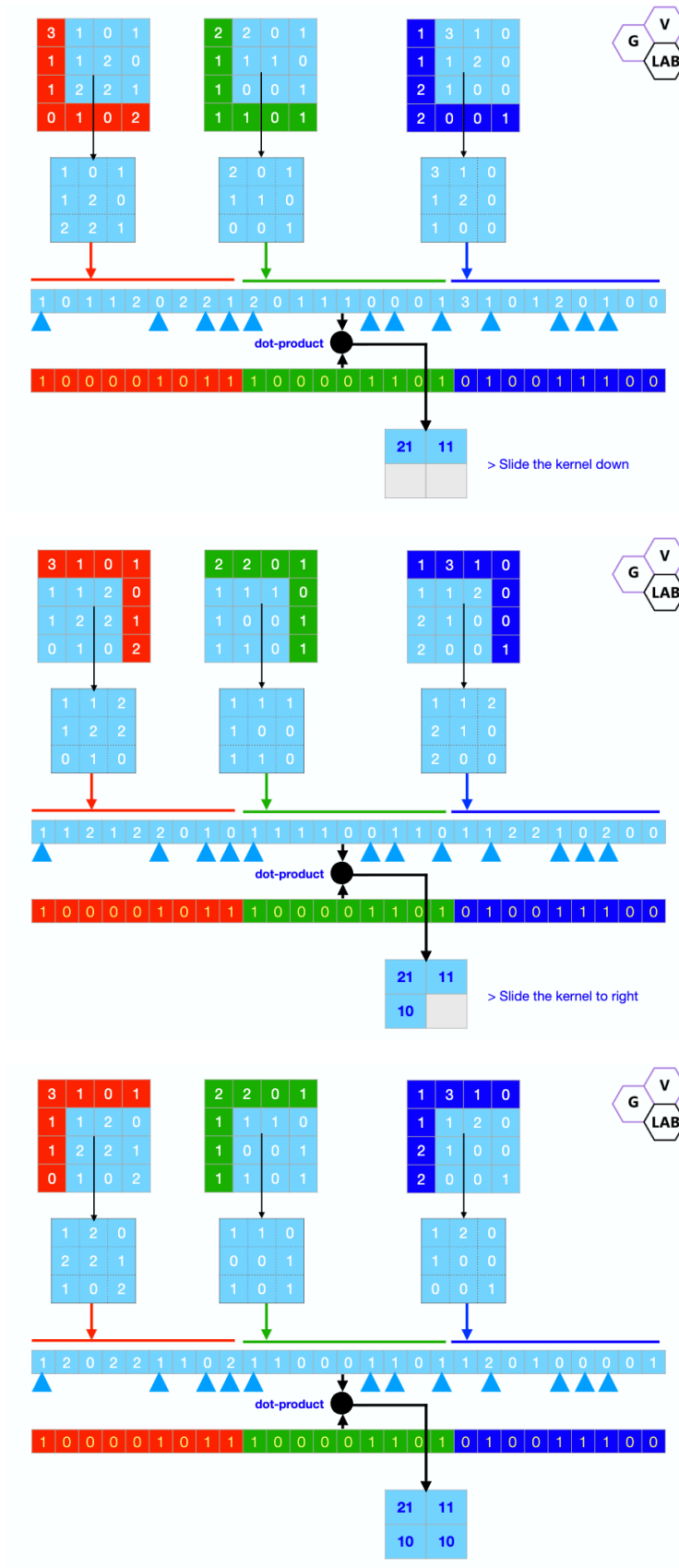
2	2	0	1
1	1	1	0
1	0	0	1
1	1	0	1

2	2	0
1	1	1
1	0	0
1	0	0

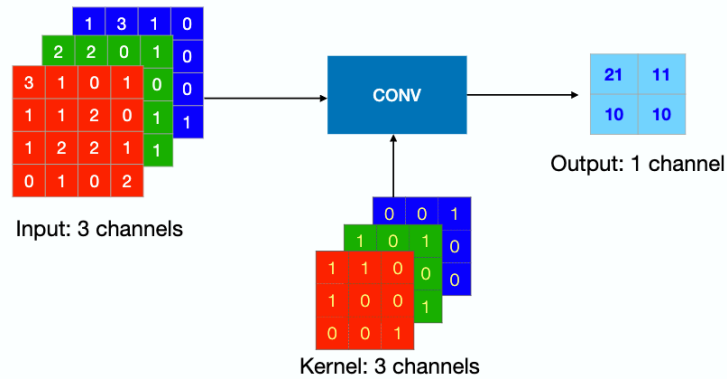
1	3	1	0
1	1	2	0
2	1	0	0
2	0	0	1

1	3	1
1	1	2
2	1	0
2	1	0





Final result



[Cảnh ảnh]: Sơ đồ 3D: input $4 \times 4 \times 3$ + kernel $3 \times 3 \times 3$ → một scalar (rồi toàn bộ output $2 \times 2 \times 1$). (Trang 7-18 slide Conv2d-multi-channels)

5. Multi-filter Layer

5.1 Một filter = một đặc trưng

Mỗi filter học phát hiện một loại đặc trưng cụ thể:

- Filter 1: phát hiện cạnh ngang
- Filter 2: phát hiện cạnh dọc
- Filter 3: phát hiện góc
- ...

Để học **nhiều loại đặc trưng**, ta dùng **nhiều filters**.

5.2 Kiến trúc Conv Layer với K filters

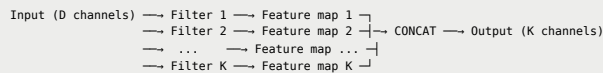
Input: feature map $i_1 \times i_2 \times D$ (D channels)

Mỗi filter f ($f = 1, \dots, K$):

- Kernel: $k_1 \times k_2 \times D$ (D channels, phải khớp với input)
- Output: một feature map $o_1 \times o_2 \times 1$

Concat K outputs theo chiều channel:

$$\text{Output} = \text{Concat}(Y_1, Y_2, \dots, Y_K) \in \mathbb{R}^{o_1 \times o_2 \times K}$$



5.3 Số tham số của một Conv Layer

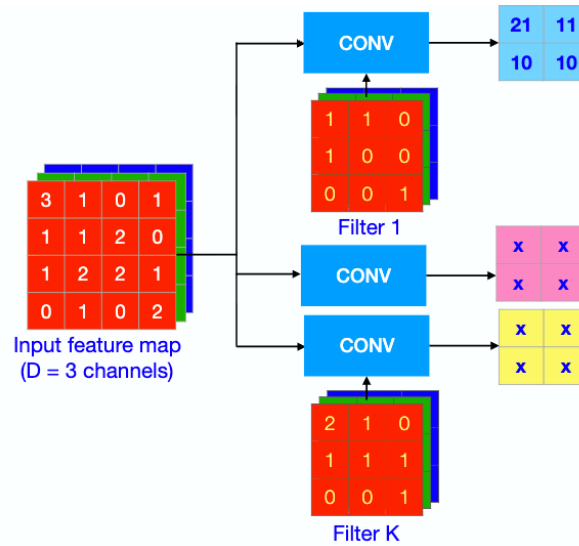
Mỗi filter: $k_1 \times k_2 \times D$ trọng số +1 bias

Tổng cho K filters:

$$\text{Params} = K \times (k_1 \times k_2 \times D + 1)$$

So với FC: FC với input $224 \times 224 \times 3 = 150,528$ và output K neuron cần $150,528 \times K$ tham số. Conv 3×3 với K filters chỉ cần $K \times (3 \times 3 \times 3 + 1) = 28K$ tham số — **giảm hơn 5000 lần** nhờ weight sharing!

Key insight: Output của Conv layer với K filters là input cho lớp tiếp theo với $D' = K$ channels. Số channels tăng lên qua các lớp Conv → mạng học ngày càng nhiều loại đặc trưng.



🖼️ [Cần ảnh]: Sơ đồ K filters → K feature maps → CONCAT → output tensor $o_1 \times o_2 \times K$. (Trang 20–23 slide Conv2d-multi-channels)

6. Padding

6.1 Vấn đề khi không padding

Với no padding: output nhỏ hơn input — $(i - k + 1) < i$ khi $k > 1$.

Sau nhiều lớp Conv, feature map nhỏ dần đi nhanh chóng. Hơn nữa, các pixel ở **biên** của ảnh chỉ tham gia vào ít vị trí tính toán hơn pixel ở giữa → mất thông tin biên.

Giải pháp: Thêm viền zero xung quanh input (zero padding) trước khi tính convolution.

6.2 Half Padding (Same Padding)

Mục tiêu: Giữ cho output có **cùng kích thước** với input.

$$p = \left\lceil \frac{k}{2} \right\rceil$$

Với kernel $k \times k$, padding p pixel ở mỗi cạnh (trái, phải, trên, dưới):

- Kernel 3×3 : $p = \lceil 3/2 \rceil = 1 \rightarrow$ thêm 1 pixel viền
- Kernel 5×5 : $p = \lceil 5/2 \rceil = 2 \rightarrow$ thêm 2 pixel viền

Kích thước sau padding: $(i_1 + 2p) \times (i_2 + 2p)$

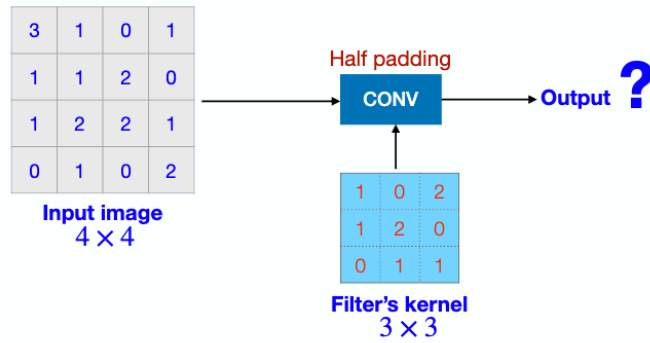
Kích thước output (stride = 1):

$$o = (i + 2p) - k + 1 = i + 2\lceil k/2 \rceil - k + 1$$

Với k lẻ: $o = i + (k - 1) - k + 1 = i$ — **output bằng input**.

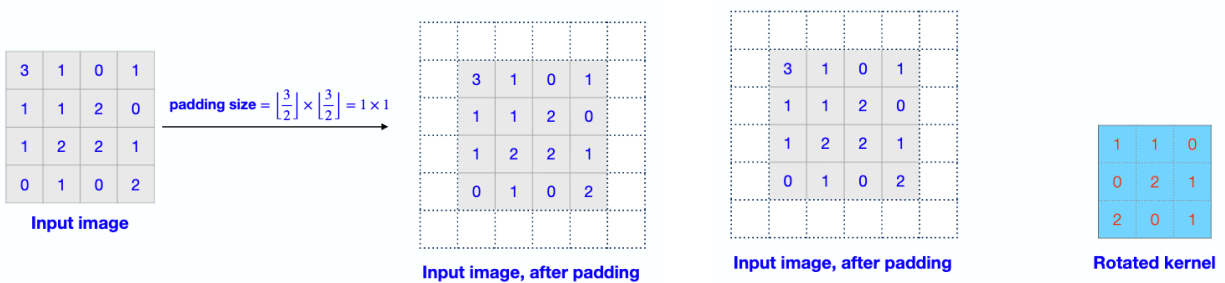
Ví dụ từ slide: Input 4×4 , Kernel 3×3 , Half padding $p = 1$:

- Input sau padding: 6×6
- Output: $(6 - 3 + 1) \times (6 - 3 + 1) = 4 \times 4$ — cùng kích thước với input ban đầu

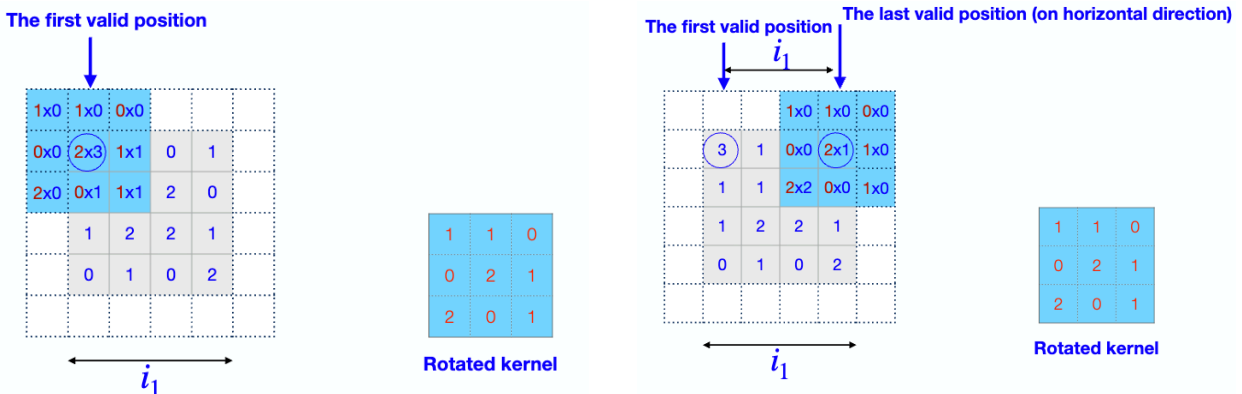


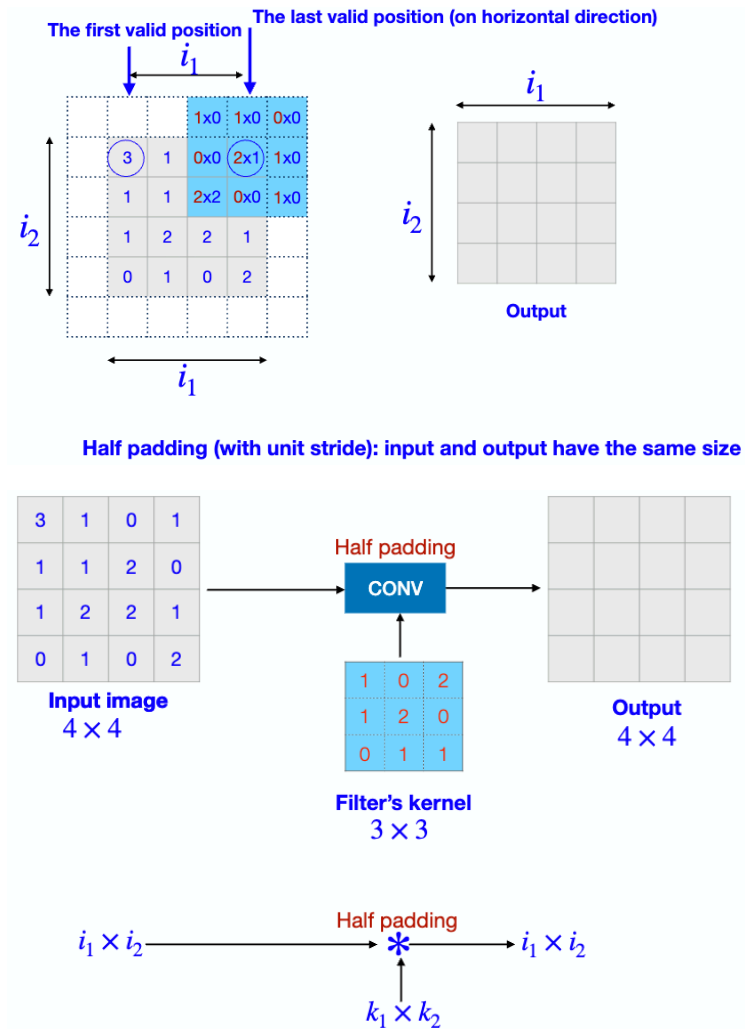
$$\text{padding size} = \left\lfloor \frac{3}{2} \right\rfloor \times \left\lfloor \frac{3}{2} \right\rfloor = 1 \times 1$$

The input is enlarged one pixel on left, right, top, and bottom

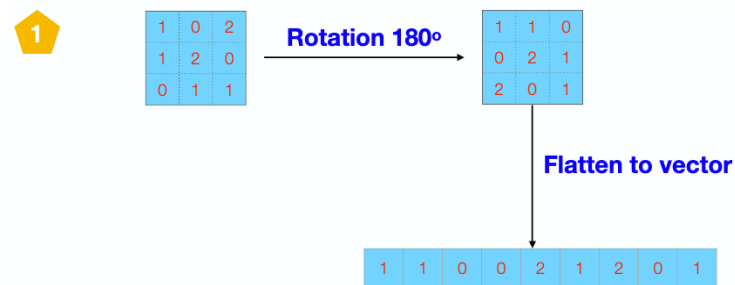


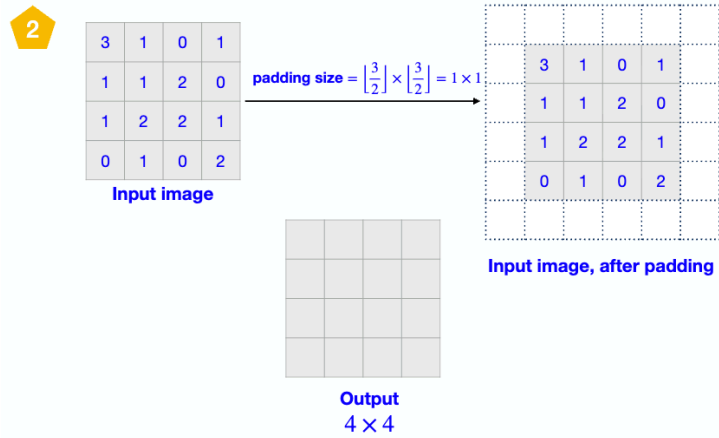
Output size = ?



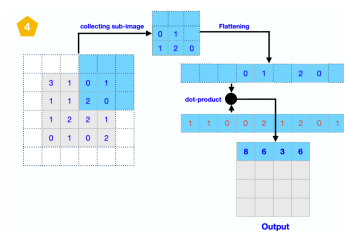
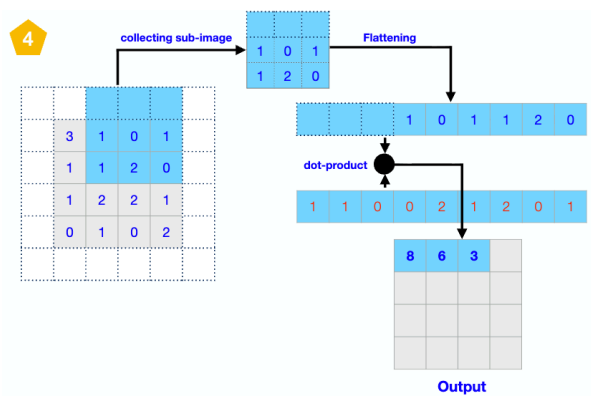
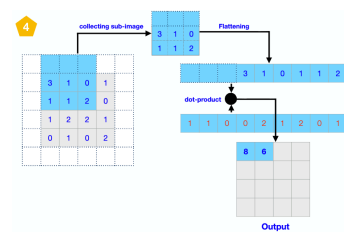
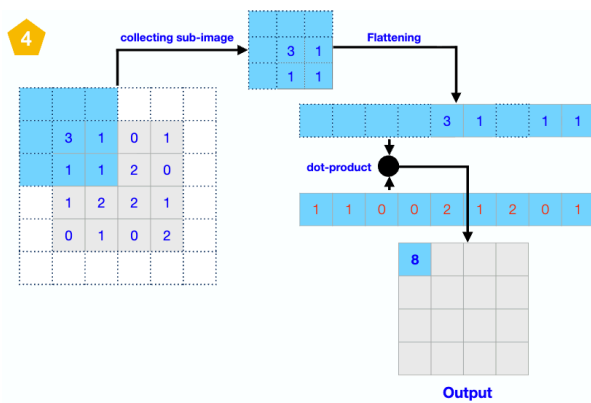
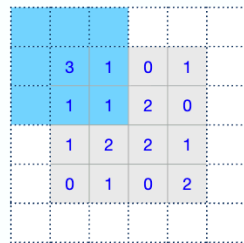


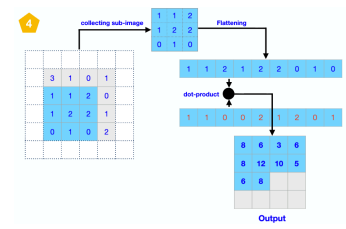
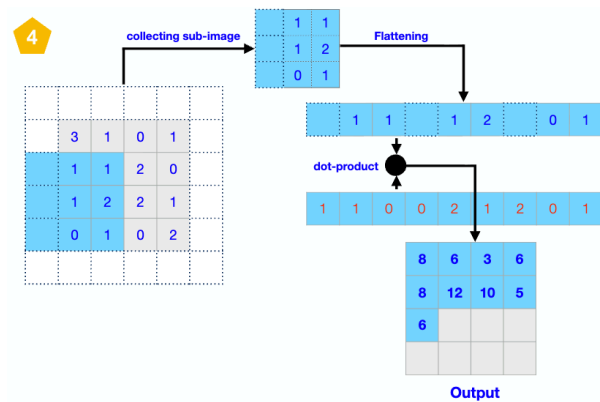
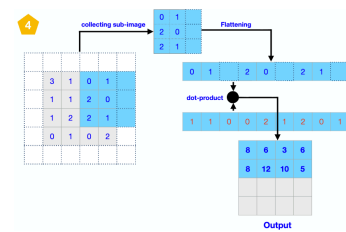
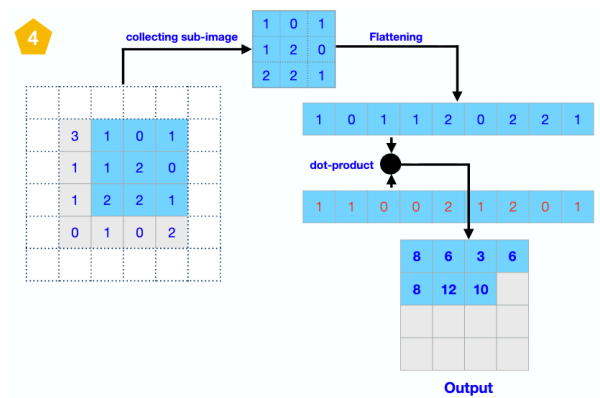
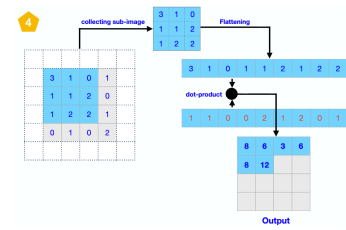
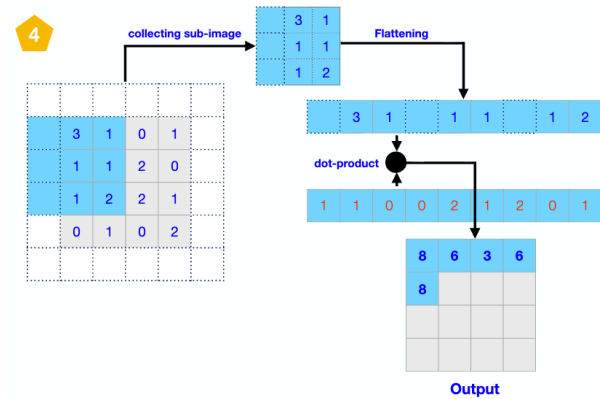
🖼️ [Cần ảnh]: Sơ đồ half padding: input 4×4 , viền zero xanh lá 1px, kernel 3×3 trượt, output 4×4 cùng kích thước. (Trang 5-32 slide Conv2d-padding)

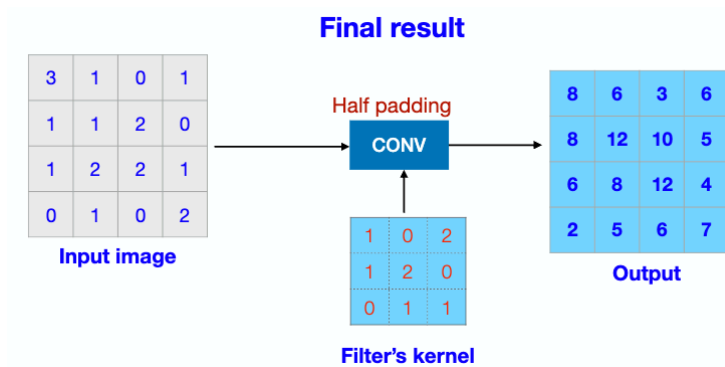
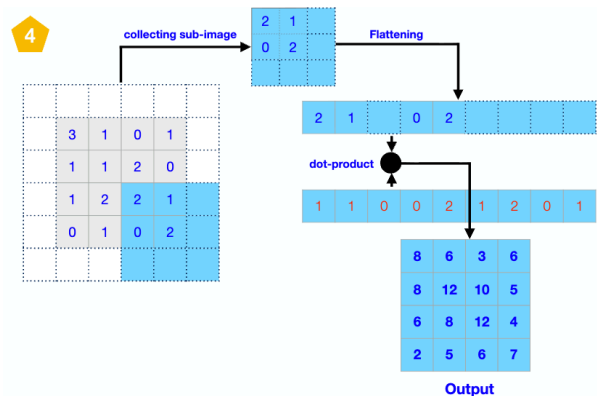
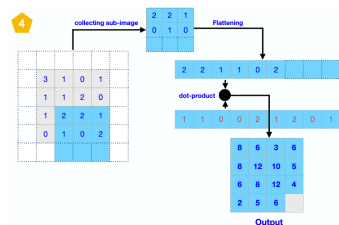
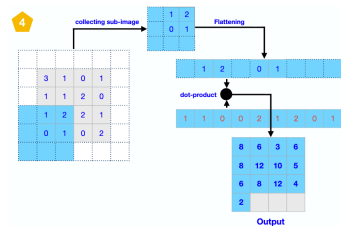
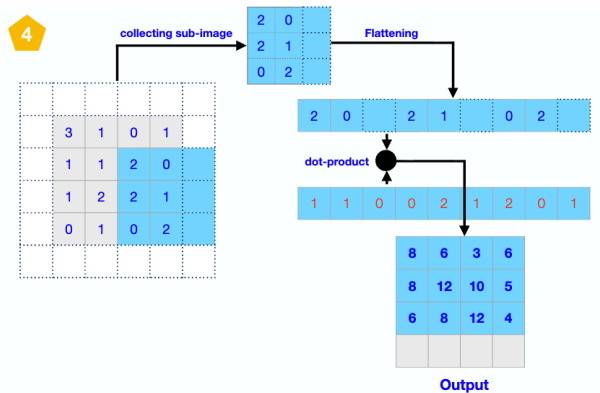




3 starting the cross-correlation process







6.3 Full Padding

Mục tiêu: Mọi pixel của input (kể cả góc, biên) đều được kernel phủ đủ lần — output **lớn hơn** input.

$$p = k - 1$$

Với kernel 3×3 : $p = 2 \rightarrow$ thêm 2 pixel viền mỗi cạnh.

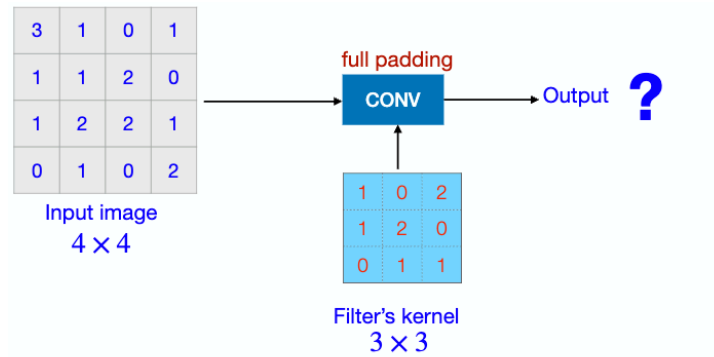
Kích thước output:

$$o_1 \times o_2 = (i_1 + k_1 - 1) \times (i_2 + k_2 - 1)$$

Ví dụ từ slide: Input 4×4 , Kernel 3×3 , Full padding $p = 2$:

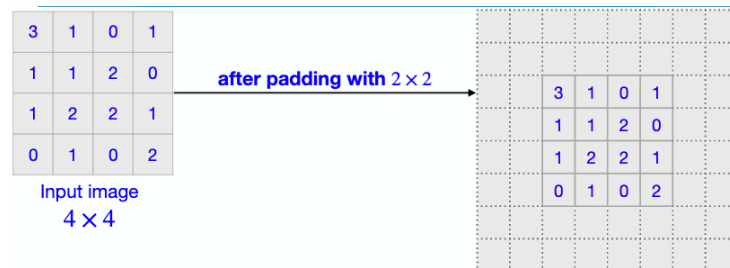
- Input sau padding: 8×8
- Output: $(8 - 3 + 1) \times (8 - 3 + 1) = 6 \times 6$ — lớn hơn input ban đầu

Khi nào dùng Full Padding? Hiếm trong CNN thông thường; thường gặp trong thiết kế đặc biệt (ví dụ: transposed convolution trong UNet decoder).



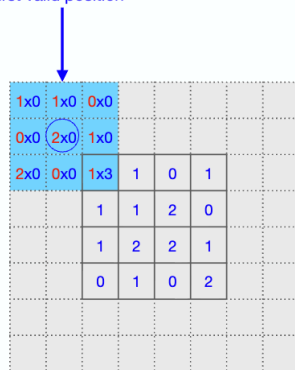
$$\text{padding size} = (3 - 1) \times (3 - 1) = 2 \times 2$$

The input is enlarged two pixel on left, right, top, and bottom



Output size = ?

The first valid position

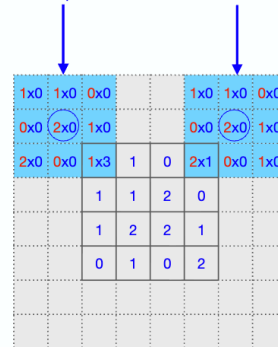


Rotated kernel

1	1	0
0	2	1
2	0	1

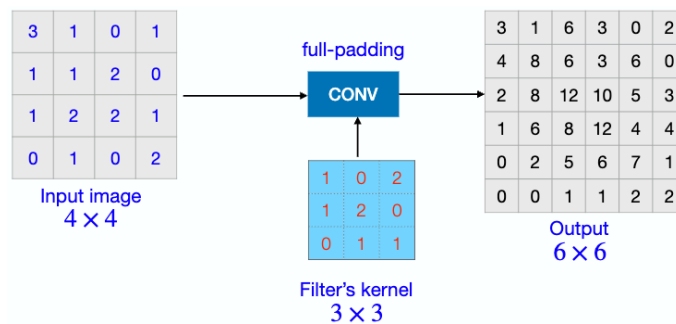
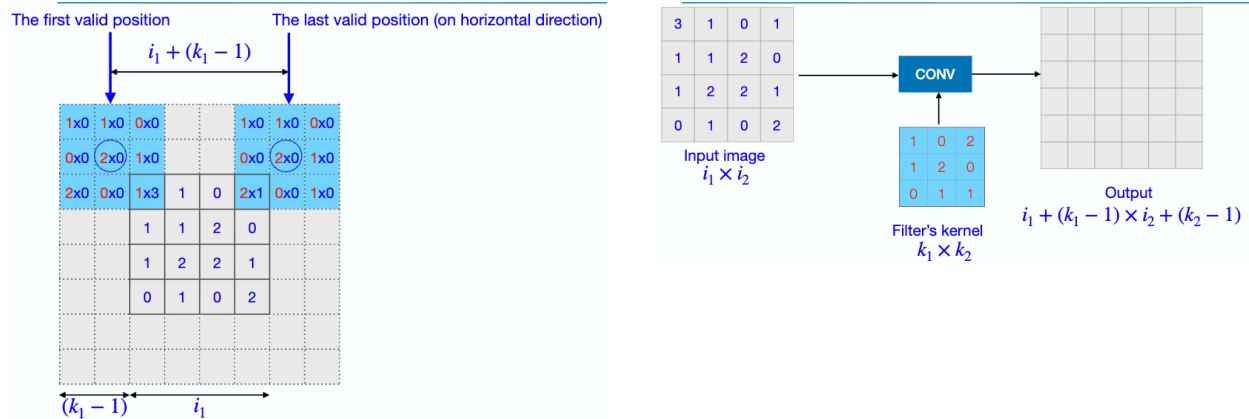
The first valid position

The last valid position (on horizontal direction)



Rotated kernel

1	1	0
0	2	1
2	0	1



The computation is the same as the example in half padding

6.4 So sánh ba chế độ Padding

Padding	p	Output size	Khi nào dùng
No padding	0	$(i - k + 1)$	Giảm size nhanh
Half (Same)	$\lfloor k/2 \rfloor$	i (giữ nguyên)	Phổ biến nhất trong CNN
Full	$k - 1$	$(i + k - 1)$	Đặc biệt, hiếm dùng

Lưu ý trong PyTorch: `padding='same'` tự động tính half padding; `padding=0` là no padding; `padding=n` là thêm n pixel viền.

7. Strides

7.1 Stride là gì?

Mặc định, kernel trượt **1 pixel** mỗi lần (stride = 1). **Stride** $s > 1$ nghĩa là kernel nhảy s pixel sau mỗi bước.

Tác dụng:

- Giảm kích thước output → **giảm tính toán** ở các lớp tiếp theo
- Tăng receptive field — mỗi neuron output "nhìn" vùng rộng hơn của input

7.2 Kích thước output với Stride

$$o_1 = \left\lfloor \frac{i_1 + 2p_1 - k_1}{s_1} \right\rfloor + 1, \quad o_2 = \left\lfloor \frac{i_2 + 2p_2 - k_2}{s_2} \right\rfloor + 1$$

Ví dụ từ slide: Input 4×4 , Kernel 3×3 , Full padding $p = 2$, Stride $s = 2$:

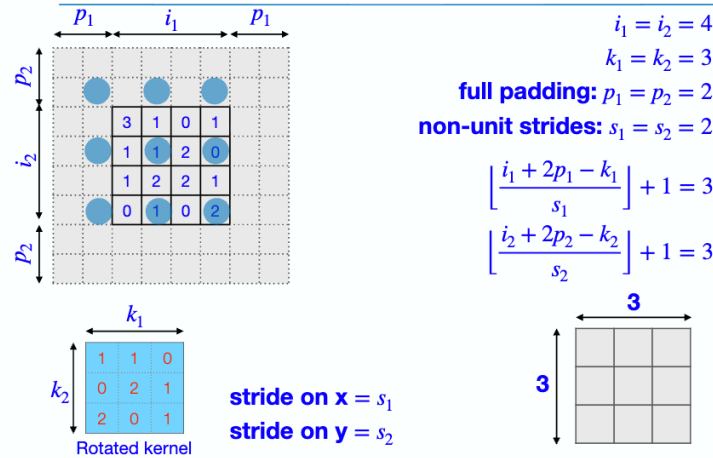
$$o = \left\lfloor \frac{4 + 2 \times 2 - 3}{2} \right\rfloor + 1 = \left\lfloor \frac{5}{2} \right\rfloor + 1 = 2 + 1 = 3$$

Output: 3×3

- Input 6×6 , Kernel 3×3 , No padding:
 - Stride=1: output 4×4 (4 vị trí theo mỗi chiều)
 - Stride=2: output 2×2 (2 vị trí theo mỗi chiều)

- Stride 1: giữ resolution, phổ biến nhất
- Stride 2: giảm resolution gấp đôi — thay thế cho Max Pooling trong một số kiến trúc hiện đại





🎥 [Cần ảnh]: Animation kernel trượt với stride=2: kernel nhảy 2 pixel mỗi bước thay vì 1. (Trang 44–52 slide Conv2d-padding)

8. Công thức Kích thước Tổng quát

8.1 Công thức tổng quát

Cho:

- Input: $i_1 \times i_2$
- Kernel: $k_1 \times k_2$
- Padding: p_1, p_2
- Stride: s_1, s_2

$$o_1 = \left\lfloor \frac{i_1 + 2p_1 - k_1}{s_1} \right\rfloor + 1, \quad o_2 = \left\lfloor \frac{i_2 + 2p_2 - k_2}{s_2} \right\rfloor + 1$$

8.2 Bảng kiểm tra nhanh (Input 4×4 , Kernel 3×3)

Padding	Stride	Output size	Công thức kiểm chứng
$p = 0$	$s = 1$	2×2	$\lfloor (4 + 0 - 3)/1 \rfloor + 1 = 2$
$p = 1$ (half)	$s = 1$	4×4	$\lfloor (4 + 2 - 3)/1 \rfloor + 1 = 4$
$p = 2$ (full)	$s = 1$	6×6	$\lfloor (4 + 4 - 3)/1 \rfloor + 1 = 6$
$p = 1$ (half)	$s = 2$	2×2	$\lfloor (4 + 2 - 3)/2 \rfloor + 1 = 2$
$p = 2$ (full)	$s = 2$	3×3	$\lfloor (4 + 4 - 3)/2 \rfloor + 1 = 3$

9. Pooling Layer

9.1 Mục đích của Pooling

Pooling layer được đặt sau Conv layer để:

- Lấy mẫu không gian đặc trưng (spatial subsampling):** Loại bỏ thông tin dư thừa — các pixel gần nhau trong feature map thường mang thông tin tương tự
- Giảm kích thước feature map** → giảm tính toán và bộ nhớ cho các lớp tiếp theo
- Bất biến dịch chuyển nhỏ (translation invariance):** Nếu đặc trưng dịch chuyển vài pixel, max pooling vẫn cho kết quả tương tự
- Giảm nhẹ overfitting** (tác dụng phụ tích cực do giảm số tham số hiệu quả)

Phép ẩn dụ: Pooling giống như "tóm tắt" một vùng của feature map thành một giá trị đại diện. Thay vì lưu từng pixel chi tiết, ta hỏi: "Trong vùng 3×3 này, đặc trưng mạnh nhất là bao nhiêu?"

9.2 Hyperparameters của Pooling

- Loại pooling:** Max, Average, Min, ...
- Window size:** kích thước cửa sổ (thường 2×2 hoặc 3×3)
- Stride:** bước trượt (thường bằng window size để không overlap)

Lưu ý quan trọng: Pooling **không có tham số học được** — không có W hay b . Đây là phép toán cố định.

9.3 Max Pooling

Cơ chế: Tại mỗi vị trí của cửa sổ, lấy **giá trị lớn nhất** trong cửa sổ đó.

Ví dụ (từ slide): Input 6×6 , Window 3×3 , Stride = 1

$$\text{Input} = \begin{bmatrix} 1 & 3 & 6 & 1 & 3 & 2 \\ 8 & 0 & 2 & 1 & 1 & 7 \\ 1 & 2 & 2 & 4 & 6 & 2 \\ 1 & 4 & 3 & 5 & 2 & 7 \\ 1 & 2 & 1 & 5 & 6 & 8 \\ 2 & 2 & 7 & 4 & 2 & 2 \end{bmatrix}$$

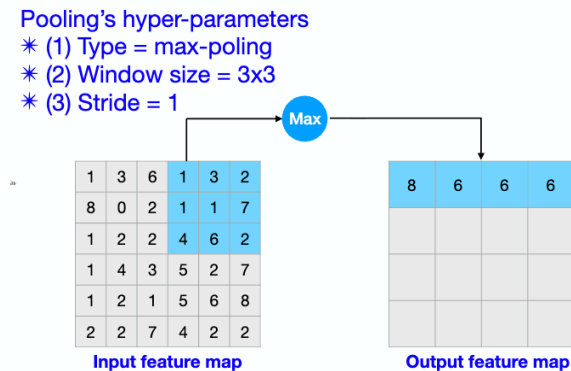
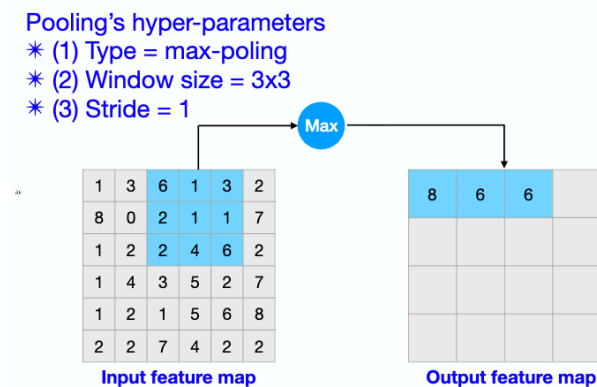
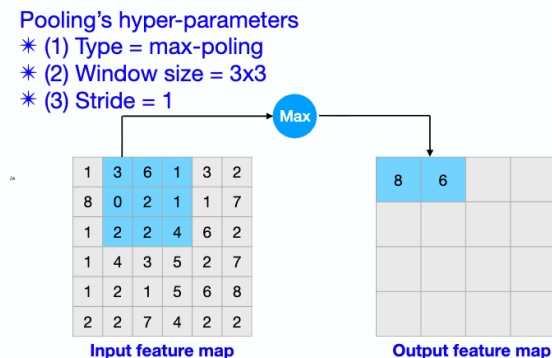
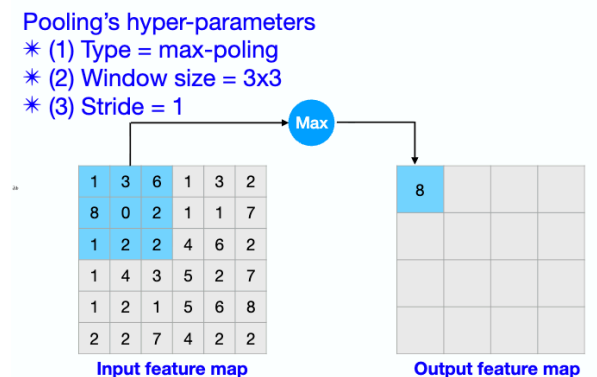
Với window 3×3 , stride=1 $\rightarrow$ Output 4×4 :

$$\text{Output} = \begin{bmatrix} 8 & 6 & 6 & 6 \\ 8 & 5 & 6 & 7 \\ 4 & 5 & 6 & 8 \\ 7 & 7 & 7 & 7 \end{bmatrix}$$

(Vị trí (1,1): max của vùng [1, 3, 6; 8, 0, 2; 1, 2, 2] = 8)

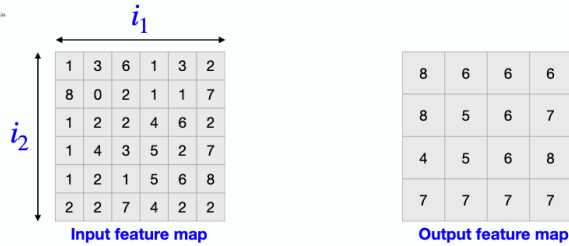
Tại sao Max Pooling phổ biến nhất?

Max pooling giữ lại **đặc trưng mạnh nhất** (giá trị cao) — nếu một filter phát hiện "có cạnh ở đâu đó trong vùng này", max pooling giữ lại tín hiệu đó dù chính xác vị trí có lệch vài pixel.



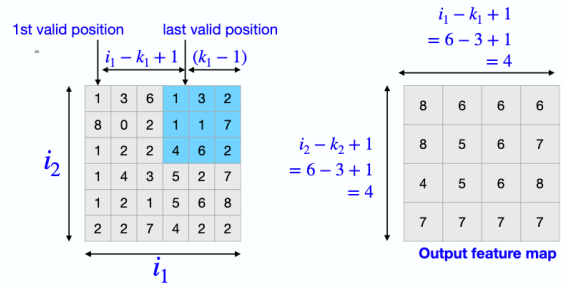
Pooling's hyper-parameters

- * (1) Type = max-pooling
- * (2) Window size = 3x3
- * (3) Stride = 1



Pooling's hyper-parameters

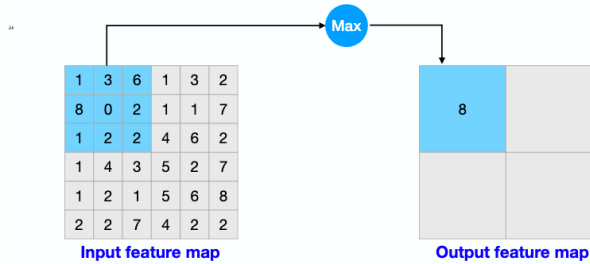
- * (1) Type = max-pooling
- * (2) Window size = 3x3
- * (3) Stride = 1



Non-unit strides

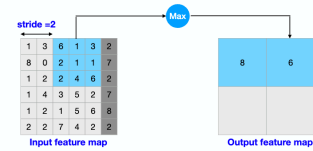
Pooling's hyper-parameters

- * (1) Type = max-pooling
- * (2) Window size = 3x3
- * (3) Stride = 2



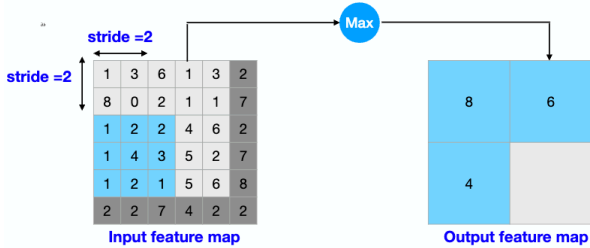
Pooling's hyper-parameters

- * (1) Type = max-pooling
- * (2) Window size = 3x3
- * (3) Stride = 2



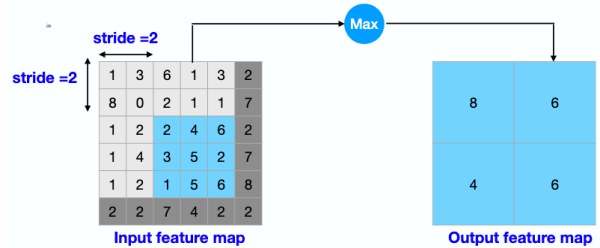
Pooling's hyper-parameters

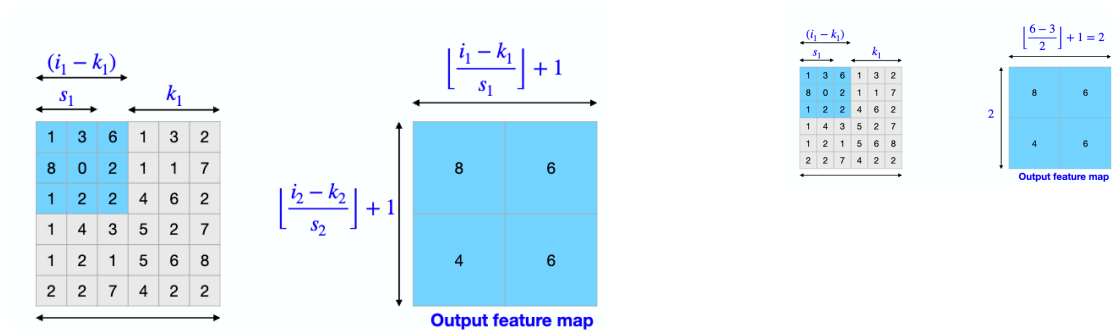
- * (1) Type = max-pooling
- * (2) Window size = 3x3
- * (3) Stride = 2



Pooling's hyper-parameters

- * (1) Type = max-pooling
- * (2) Window size = 3x3
- * (3) Stride = 2





9.4 Kích thước Output của Pooling

Không padding (pooling thường không dùng padding):

$$o_1 = \left\lfloor \frac{i_1 - k_1}{s_1} \right\rfloor + 1, \quad o_2 = \left\lfloor \frac{i_2 - k_2}{s_2} \right\rfloor + 1$$

Ví dụ: Input 6×6 , Window 3×3 , Stride=2:

$$o = \left\lfloor \frac{6 - 3}{2} \right\rfloor + 1 = \lfloor 1.5 \rfloor + 1 = 1 + 1 = 2$$

Output: 2×2 — giảm từ 6×6 xuống 2×2 chỉ sau một lớp pooling!

Cấu hình phổ biến: Window 2×2 , Stride=2 — giảm $\frac{1}{2}$ mỗi chiều (diện tích giảm 4 lần):

$$o = \left\lfloor \frac{i - 2}{2} \right\rfloor + 1 = \frac{i}{2}$$

9.5 Các loại Pooling khác

Average Pooling

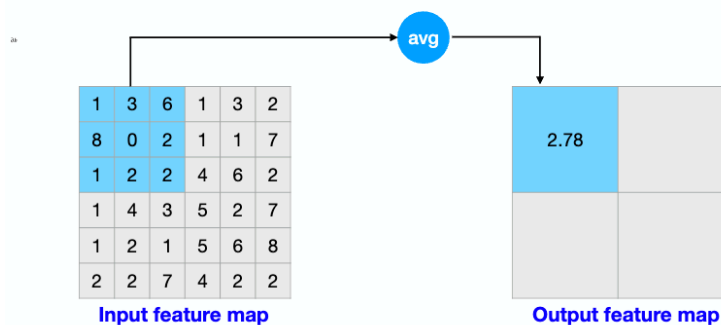
Lấy giá trị trung bình trong cửa sổ:

$$Y(u, v) = \frac{1}{k_1 \times k_2} \sum_{i=0}^{k_1-1} \sum_{j=0}^{k_2-1} X(u \cdot s + i, v \cdot s + j)$$

- Mượt hơn max pooling (không bị ảnh hưởng bởi outlier trong cửa sổ)
- Global Average Pooling (GAP):** Window = toàn bộ feature map → một scalar per channel — dùng thay FC layer ở cuối nhiều CNN hiện đại (ResNet, EfficientNet)

Pooling's hyper-parameters

- * (1) Type = **average**
- * (2) Window size = 3×3
- * (3) Stride = 2

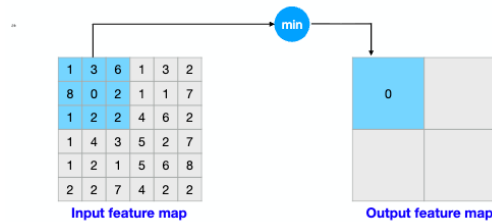


Min Pooling

Lấy **giá trị nhỏ nhất** — ít phổ biến, đôi khi dùng trong preprocessing đặc biệt

Pooling's hyper-parameters

- * (1) Type = **min**
- * (2) Window size = 3x3
- * (3) Stride = 2



So sánh

Loại	Cơ chế	Ưu điểm	Khi nào dùng
Max	max trong window	Giữ đặc trưng mạnh, translation invariance	Phổ biến nhất, phân loại
Average	mean trong window	Mượt hơn, không bỏ thông tin	GAP ở cuối mạng; feature aggregation
Min	min trong window	Tìm vùng tối/nền	Rất hiếm

10. Tổng kết

10.1 So sánh Conv Layer vs Pooling Layer

Đặc điểm	Conv Layer	Pooling Layer
Tham số	Có (W, b) — học được	Không — cố định
Mục đích	Trích xuất đặc trưng	Giảm kích thước
Output channels	K' (số filters)	Giống input
Size thay đổi?	Phụ thuộc padding	Giảm theo stride

10.2 Pipeline CNN điển hình



10.3 Các công thức cần nhớ

$$o = \left\lfloor \frac{i + 2p - k}{s} \right\rfloor + 1$$

Tình huống	Công thức rút gọn
No padding, stride=1	$o = i - k + 1$
Same/Half padding, stride=1	$o = i$
Full padding, stride=1	$o = i + k - 1$
Max pooling no padding	$o = \lfloor (i - k) / s \rfloor + 1$

10.4 Key Insights

Convolution trong DL thực ra là Cross-Correlation — kernel không flip, tên gọi chỉ là quy ước. Không ảnh hưởng vì kernel là tham số học được. **Weight sharing là sức mạnh cốt lõi của CNN**: Cùng filter áp lên mọi vị trí → nhận dạng đặc trưng bất kể vị trí (translation equivariance). **Một filter đa-channel → một feature map**: Để có $K \times K$ feature maps, cần $K \times K$ filters. Output của Conv layer có $K \times K$ channels. **Padding = Same** là lựa chọn phổ biến nhất vì giữ kích thước, tránh mất thông tin biên. **Stride 2 thay thế MaxPool** trong nhiều kiến trúc hiện đại (ResNet, EfficientNet): học cách giảm kích thước thay vì dùng max cố định. **Không có tham số trong Pooling** → không backprop qua pooling như Conv. Max pooling chỉ truyền gradient về vị trí có giá trị max (gradient routing).

11. Bài Tập Tính Toán

Làm hết các bài tập này trước khi xem đáp án — đây là dạng bài thi thường gặp.

Bài 1 — Tính kích thước output (5 câu nhỏ)

Cho mỗi cấu hình sau, tính kích thước output $O_1 \times O_2$:

Câu	Input	Kernel	Padding	Stride
a	7×7	3×3	0	1
b	7×7	3×3	1	1
c	7×7	3×3	0	2
d	32×32	5×5	2	1
e	28×28	3×3	1	2

▼ Đáp án

Dùng công thức: $O = \lfloor (i + 2p - k) / s \rfloor + 1$

Câu	Tính	Output
a	$\lfloor (7 + 0 - 3) / 1 \rfloor + 1 = 5$	5×5
b	$\lfloor (7 + 2 - 3) / 1 \rfloor + 1 = 7$	7×7 — same padding giữ nguyên kích thước ✓
c	$\lfloor (7 + 0 - 3) / 2 \rfloor + 1 = \lfloor 4/2 \rfloor + 1 = 3$	3×3
d	$\lfloor (32 + 4 - 5) / 1 \rfloor + 1 = 32$	32×32 — same padding với $k=5, p=2$ ✓
e	$\lfloor (28 + 2 - 3) / 2 \rfloor + 1 = \lfloor 27/2 \rfloor + 1 = 14$	14×14

Bài 2 — Tính tích chập thủ công

Cho input X và kernel W :

$$X = \begin{bmatrix} 1 & 2 & 3 \\ 0 & 1 & 2 \\ 1 & 0 & 1 \end{bmatrix}, \quad W = \begin{bmatrix} 1 & 0 \\ -1 & 2 \end{bmatrix}$$

(a) Tính output Y của **cross-correlation** (PyTorch `Conv2d`), no padding, stride=1.

(b) Tính output nếu dùng **convolution** thực sự (flip W 180° trước).

(c) Kích thước output là bao nhiêu?

▼ Đáp án Bài 2

(c) **Kích thước**: $O = (3 - 2 + 1) \times (3 - 2 + 1) = 2 \times 2$

(a) **Cross-correlation** (không flip W):

Tại mỗi vị trí, lấy sub-matrix 2×2 và tính dot product với $W = \begin{bmatrix} 1, 0 \\ -1, 2 \end{bmatrix}$:

Vị trí	Sub-matrix	Dot product
(0,0)	$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$	$1 \times 1 + 2 \times 0 + 0 \times (-1) + 1 \times 2 = 1 + 0 + 0 + 2 = \mathbf{3}$
(0,1)	$\begin{bmatrix} 2 & 3 \\ 1 & 2 \end{bmatrix}$	$2 \times 1 + 3 \times 0 + 1 \times (-1) + 2 \times 2 = 2 + 0 - 1 + 4 = \mathbf{5}$
(1,0)	$\begin{bmatrix} 0 & 1 \\ 1 & 0 \end{bmatrix}$	$0 \times 1 + 1 \times 0 + 1 \times (-1) + 0 \times 2 = 0 + 0 - 1 + 0 = \mathbf{-1}$
(1,1)	$\begin{bmatrix} 1 & 2 \\ 0 & 1 \end{bmatrix}$	$1 \times 1 + 2 \times 0 + 0 \times (-1) + 1 \times 2 = 1 + 0 + 0 + 2 = \mathbf{3}$

$$Y_{\text{corr}} = \begin{bmatrix} 3 & 5 \\ -1 & 3 \end{bmatrix}$$

(b) **Convolution thực sự** — flip W 180° trước:

$$\text{Rot}_{180^\circ}(W) = \begin{bmatrix} 2 & -1 \\ 0 & 1 \end{bmatrix}$$

Vị trí	Dot product với $\text{Rot}(W)$
(0,0)	$1 \times 2 + 2 \times (-1) + 0 \times 0 + 1 \times 1 = 2 - 2 + 0 + 1 = 1$
(0,1)	$2 \times 2 + 3 \times (-1) + 1 \times 0 + 2 \times 1 = 4 - 3 + 0 + 2 = 3$
(1,0)	$0 \times 2 + 1 \times (-1) + 1 \times 0 + 0 \times 1 = 0 - 1 + 0 + 0 = -1$
(1,1)	$1 \times 2 + 2 \times (-1) + 0 \times 0 + 1 \times 1 = 2 - 2 + 0 + 1 = 1$

$$Y_{conv} = \begin{bmatrix} 1 & 3 \\ -1 & 1 \end{bmatrix}$$

Nhận xét: $Y_{xcorr} \neq Y_{conv}$ — hai phép toán khác nhau. PyTorch `Conv2d` thực hiện cross-correlation nhưng gọi là "convolution".

Bài 3 — Đếm tham số Conv Layer

Một Conv layer có:

- Input: feature map $28 \times 28 \times 32$ (32 channels)
- Kernel size: 3×3
- Số filters: $K = 64$
- Padding: 1, Stride: 1
- Có bias

(a) Tính số tham số (weights + bias) của layer này.

(b) Kích thước output tensor là bao nhiêu?

(c) Nếu thay bằng Fully Connected layer (input $28 \times 28 \times 32 \rightarrow$ output $28 \times 28 \times 64$), cần bao nhiêu tham số? So sánh với câu (a).

▼ Đáp án Bài 3

(a) **Số tham số Conv:**

Mỗi filter: $k \times k \times D_{in} + 1 = 3 \times 3 \times 32 + 1 = 288 + 1 = 289$ tham số

Tổng $K = 64$ filters:

$$\text{Params}_{Conv} = 64 \times 289 = \mathbf{18,496}$$

(b) **Kích thước output:**

$$o = \lfloor (28 + 2 \times 1 - 3) / 1 \rfloor + 1 = 28$$

Output: $28 \times 28 \times 64$

(c) **So sánh với FC:**

FC input size: $28 \times 28 \times 32 = 25,088$

FC output size: $28 \times 28 \times 64 = 50,176$

FC params (với bias): $25,088 \times 50,176 + 50,176 \approx \mathbf{1,259,599,488}$ (~1.26 tỷ!)

$$\text{Tỉ lệ} = \frac{1,259,599,488}{18,496} \approx \mathbf{68,100 \times}$$

→ Conv layer dùng ít hơn FC khoảng 68,000 lần nhờ weight sharing!

Bài 4 — Multi-channel convolution

Input ảnh RGB $5 \times 5 \times 3$. Dùng kernel $3 \times 3 \times 3$ như sau:

- Channel R: $W_R = \begin{bmatrix} 1 & 0 & -1 \\ 1 & 0 & -1 \\ 1 & 0 & -1 \end{bmatrix}$ (Sobel dọc)
- Channel G: $W_G = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ (zeros)
- Channel B: $W_B = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$ (zeros)

Input tại vùng 3×3 góc trên trái:

$$X_R = \begin{bmatrix} 255 & 200 & 100 \\ 255 & 200 & 100 \\ 255 & 200 & 100 \end{bmatrix}, \quad X_G = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}, \quad X_B = \begin{bmatrix} 0 & 0 & 0 \\ 0 & 0 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$

(a) Tính giá trị output $Y(0,0)$ tại góc trên trái (cross-correlation, no padding).

(b) Kernel này phát hiện đặc trưng gì? Tại sao?

▼ **Đáp án Bài 4**

(a) Tính $Y(0, 0)$:

$$Y(0, 0) = \sum_{c \in \{R, G, B\}} \text{dot}(X_c[0 : 3, 0 : 3], W_c)$$

Đóng góp từ G và B = 0 (kernel zeros).

Đóng góp từ R:

$$\text{dot}(X_R, W_R) = \sum_{i,j} X_R(i, j) \times W_R(i, j)$$

$$= 255 \times 1 + 200 \times 0 + 100 \times (-1) + 255 \times 1 + 200 \times 0 + 100 \times (-1) + 255 \times 1 + 200 \times 0 + 100 \times (-1) = (255 - 100) \times 3 = 155 \times 3 = \mathbf{465}$$

(b) Kernel này phát hiện:

Cạnh dọc trong channel Red. Kernel Sobel dọc W_R cho giá trị dương khi cột trái (pixel sáng, R=255) > cột phải (pixel tối, R=100). Giá trị 465 lớn → có sự thay đổi mạnh theo chiều ngang trong kênh đỏ → phát hiện biên dọc.

Nếu ảnh đồng nhất (R toàn 255): $255 \times 1 + 255 \times 0 + 255 \times (-1) = 0 \rightarrow$ không phát hiện cạnh.

Bài 5 — Padding và Stride kết hợp

Một mạng CNN có 3 Conv layers liên tiếp:

Layer	Kernel	Padding	Stride	Filters
Conv1	3×3	1	1	32
Conv2	3×3	0	2	64
Conv3	5×5	2	1	128

Input ban đầu: $64 \times 64 \times 3$.

(a) Tính kích thước output sau mỗi layer (dạng H×W×C).

(b) Tính tổng số tham số (weights + bias) của toàn bộ 3 layers.

▼ **Đáp án Bài 5**

(a) **Kích thước sau từng layer:**

Sau Conv1 ($i = 64, k = 3, p = 1, s = 1, K = 32$):

$$o = \lfloor (64 + 2 - 3) / 1 \rfloor + 1 = 64$$

→ **64×64×32**

Sau Conv2 ($i = 64, k = 3, p = 0, s = 2, K = 64, D_{in} = 32$):

$$o = \lfloor (64 + 0 - 3) / 2 \rfloor + 1 = \lfloor 61 / 2 \rfloor + 1 = 30 + 1 = 31$$

→ **31×31×64**

Sau Conv3 ($i = 31, k = 5, p = 2, s = 1, K = 128, D_{in} = 64$):

$$o = \lfloor (31 + 4 - 5) / 1 \rfloor + 1 = 31$$

→ **31×31×128**

(b) **Tổng số tham số:**

Layer	Tham số mỗi filter	Số filters	Tổng
Conv1	$3 \times 3 \times 3 + 1 = 28$	32	$32 \times 28 = 896$
Conv2	$3 \times 3 \times 32 + 1 = 289$	64	$64 \times 289 = 18,496$
Conv3	$5 \times 5 \times 64 + 1 = 1,601$	128	$128 \times 1,601 = 204,928$

$$\text{Total} = 896 + 18,496 + 204,928 = \mathbf{224,320}$$

Nhận xét: Conv3 chiếm hơn 91% tổng tham số — kernel lớn hơn và nhiều channels hơn chi phối.

Bài 6 — Max Pooling

Cho feature map 4×4 :

$$X = \begin{bmatrix} 2 & 8 & 3 & 1 \\ 6 & 4 & 7 & 5 \\ 1 & 9 & 0 & 3 \\ 4 & 2 & 8 & 6 \end{bmatrix}$$

(a) Tính output của Max Pooling với window 2×2 , stride = 2.

(b) Tính output của Average Pooling với cùng cấu hình.

(c) Kích thước output là bao nhiêu?

▼ **Đáp án Bài 6**

(c) **Kích thước:**

$$o = \lfloor (4 - 2)/2 \rfloor + 1 = 2$$

Output: 2×2

Chia feature map thành 4 vùng 2×2 không chồng lấn:

```
Vùng TL (0:2, 0:2): [2, 8; 6, 4]
Vùng TR (0:2, 2:4): [3, 1; 7, 5]
Vùng BL (2:4, 0:2): [1, 9; 4, 2]
Vùng BR (2:4, 2:4): [0, 3; 8, 6]
```

(a) **Max Pooling:**

$$Y_{max} = \begin{bmatrix} \max(2, 8, 6, 4) & \max(3, 1, 7, 5) \\ \max(1, 9, 4, 2) & \max(0, 3, 8, 6) \end{bmatrix} = \begin{bmatrix} 8 & 7 \\ 9 & 8 \end{bmatrix}$$

(b) **Average Pooling:**

$$Y_{avg} = \begin{bmatrix} (2 + 8 + 6 + 4)/4 & (3 + 1 + 7 + 5)/4 \\ (1 + 9 + 4 + 2)/4 & (0 + 3 + 8 + 6)/4 \end{bmatrix} = \begin{bmatrix} 5.0 & 4.0 \\ 4.0 & 4.25 \end{bmatrix}$$

Nhận xét: Max pooling giữ lại giá trị nổi bật (8, 7, 9, 8), Average pooling làm mượt (5.0, 4.0, 4.0, 4.25) — Max hữu ích hơn khi muốn phát hiện "có đặc trưng này ở đâu đó trong vùng".

Bài 7 — Tracing shapes qua pipeline CNN (bài tổng hợp)

Một CNN để phân loại ảnh CIFAR-10 (10 classes) có kiến trúc:

```
Input: 32×32×3
→ Conv(k=3, p=1, s=1, K=16) → ReLU
→ MaxPool(2×2, s=2)
→ Conv(k=3, p=1, s=1, K=32) → ReLU
→ MaxPool(2×2, s=2)
→ Conv(k=3, p=0, s=1, K=64) → ReLU
→ Global Average Pooling
→ FC(10)
→ Softmax
```

(a) Trace kích thước tensor qua từng layer.

(b) Tính tổng số tham số (3 Conv layers + 1 FC layer).

(c) Nếu không dùng Global Average Pooling mà Flatten trước FC, FC layer cần bao nhiêu tham số? So sánh với câu (b).

▼ **Đáp án Bài 7**

(a) **Shape tracing:**

Layer	Phép tính	Output shape
Input	—	32×32×3
Conv1 (k=3,p=1,s=1,K=16)	$o = \lfloor (32 + 2 - 3)/1 \rfloor + 1 = 32$	32×32×16
MaxPool (2×2, s=2)	$o = \lfloor (32 - 2)/2 \rfloor + 1 = 16$	16×16×16
Conv2 (k=3,p=1,s=1,K=32)	$o = \lfloor (16 + 2 - 3)/1 \rfloor + 1 = 16$	16×16×32
MaxPool (2×2, s=2)	$o = \lfloor (16 - 2)/2 \rfloor + 1 = 8$	8×8×32
Conv3 (k=3,p=0,s=1,K=64)	$o = \lfloor (8 + 0 - 3)/1 \rfloor + 1 = 6$	6×6×64
Global Avg Pool	mean qua H×W	1×1×64 = 64
FC(10)	linear	10
Softmax	—	10

(b) **Tổng tham số:**

Layer	Công thức	Tham số
Conv1	$16 \times (3 \times 3 \times 3 + 1)$	$16 \times 28 = 448$
Conv2	$32 \times (3 \times 3 \times 16 + 1)$	$32 \times 145 = 4,640$
Conv3	$64 \times (3 \times 3 \times 32 + 1)$	$64 \times 289 = 18,496$
FC	$64 \times 10 + 10$	650
Total		24,234

(c) **Nếu Flatten trước FC:**

Sau Conv3: $6 \times 6 \times 64 = 2,304$ features

FC: $2,304 \times 10 + 10 = 23,050$ tham số chỉ riêng FC

So sánh FC layer:

- Với GAP: 650 tham số ở FC
- Với Flatten: 23,050 tham số ở FC

→ **GAP giảm 35x số tham số ở FC layer** và giúp mô hình tổng quát hóa tốt hơn (ít overfit hơn) — lý do GAP phổ biến trong ResNet, EfficientNet.

Bài 8 — Câu hỏi tư duy nhanh

(a) Conv2d với kernel 1×1 , K filters, input D channels làm gì? Có bao nhiêu tham số? Kích thước output thay đổi không?

(b) Tại sao hai Conv 3×3 xếp chồng thường tốt hơn một Conv 5×5 về cả số tham số lẫn biểu diễn?

(c) Depthwise Separable Convolution tách Conv thành 2 bước: Depthwise Conv ($k \times k$ per channel) + Pointwise Conv (1×1). Với input $H \times W \times D$, kernel $k \times k$, output K channels, so sánh số tham số với standard Conv?

▼ Đáp án Bài 8

(a) Conv 1x1:

- Tác dụng: biến đổi tuyến tính theo chiều channel tại từng pixel riêng lẻ — tương đương FC layer áp dụng độc lập tại mỗi vị trí spatial.
- Tham số: $K \times (1 \times 1 \times D + 1) = K \times (D + 1)$
- Kích thước spatial (H×W): **không thay đổi**. Chỉ thay đổi số channels $D \rightarrow K$.
- **Ứng dụng phổ biến**: Bottleneck layers trong ResNet, head trong SegFormer/YOLO.

(b) Hai Conv 3×3 vs một Conv 5×5 :

Receptive field: Cả hai đều có receptive field cuối = 5×5 (hai lớp 3×3 tương đương một lớp 5×5 về receptive field).

Tham số (giả sử D channels vào và ra):

	Tham số
Một Conv 5×5	$D \times 5 \times 5 \times D = 25D^2$
Hai Conv 3×3	$D \times 3 \times 3 \times D + D \times 3 \times 3 \times D = 2 \times 9D^2 = 18D^2$

Tiết kiệm: $25D^2 - 18D^2 = 7D^2$, tỉ lệ: **28% ít hơn**.

Phi tuyến: Hai Conv 3×3 có hai lần ReLU → **biểu diễn phức tạp hơn** (thêm một tầng phi tuyến). Đây là lý do VGG-16 dùng hai Conv 3×3 thay vì dùng Conv lớn.

(c) Depthwise Separable vs Standard Conv:

	Standard Conv	Depthwise Separable
Depthwise	—	$D \times k \times k \times 1 = Dk^2$
Pointwise	—	$K \times 1 \times 1 \times D = KD$
Standard	$K \times k \times k \times D = Kk^2D$	—
Tổng	Kk^2D	$Dk^2 + KD = D(k^2 + K)$

Tỉ lệ tiết kiệm:

$$\frac{D(k^2 + K)}{Kk^2D} = \frac{k^2 + K}{Kk^2} = \frac{1}{K} + \frac{1}{k^2}$$

Với $k = 3$, $K = 64$: tỉ lệ = $1/64 + 1/9 \approx 0.127$ → **tiết kiệm ~8.7x** — cơ sở của MobileNet!

Tổng hợp dạng bài và công thức

Dạng bài	Công thức cốt lõi
Output size	$o = \lfloor (i + 2p - k) / s \rfloor + 1$
Same padding	$p = \lfloor k/2 \rfloor \rightarrow o = i$ (s=1, k lẻ)
Conv params	$K \times (k \times k \times D_{in} + 1)$
Cross-correlation	dot product tại mỗi vị trí, không flip kernel
Convolution	flip kernel 180° rồi cross-correlation
MaxPool output	$o = \lfloor (i - k) / s \rfloor + 1$
MaxPool 2x2, s=2	$o = i/2$ — giảm đôi mỗi chiều
Global Avg Pool	$(H, W, C) \rightarrow (1, 1, C) = C$ — không có tham số
1x1 Conv	Biến đổi channel, giữ spatial, $K \times (D + 1)$ params